

PLANO DE TESTES

ServeRest API (serverest.dev) — Testes Manuais Funcionais de API

Item	Detalhe
Autor	Luiz Vinicius Cunha Maciel
Projeto	Portfólio pessoal de QA
Versão	1.0
Data	29/07/2026

1. INTRODUÇÃO

Este Plano de Testes foi construído para a API pública ServeRest (<https://serverest.dev>), uma API REST gratuita e open source que simula o back-end de uma loja virtual, criada com o propósito de servir como ambiente de estudo e prática de testes de software, manuais e automatizados.

A ServeRest expõe recursos relacionados a usuários, produtos e carrinhos de compra, contempla regras de negócio (exclusividade de e-mail, controle de estoque, permissões de administrador, vínculo entre carrinho e usuário/produto) e possui documentação interativa (Swagger) diretamente na própria URL, o que a torna um ambiente interessante para a execução de um bom plano de testes de API.

2. ESPECIFICAÇÕES DO PROJETO

2.1. OBJETIVO GERAL

Por meio de testes manuais de API, validar as principais funcionalidades da ServeRest : cadastro, consulta, atualização e exclusão de usuários e produtos, autenticação e gestão de carrinho de compras assim garantindo que os endpoints retornem os dados, status codes e mensagens de erro esperados, identificando defeitos, inconsistências no contrato da API e comportamentos não previstos nas regras de negócio.

2.2. APLICAÇÃO SOB TESTE

Item	Detalhe
Nome da aplicação	ServeRest - https://serverest.dev
Tipo	API REST simulando back-end de e-commerce
Ambiente	Produção (ambiente público de demonstração)
Formato de dados	JSON (request e response)
Autenticação	Token bearer obtido via POST /login, enviado no header Authorization
Documentação	Swagger interativo disponível em https://serverest.dev
Ferramentas de requisição	Postman / Insomnia

2.3. FUNCIONALIDADES EM ESCOPO

Recurso /login

- POST /login -> autenticação e geração de bearer Token

Recurso /usuarios

- GET /usuarios -> listagem de usuários e filtros por query string
- GET /usuarios/{_id} -> consulta de usuário por ID
- POST /usuarios -> cadastro de novo usuário
- PUT /usuarios/{_id} -> edição de usuário existente
- DELETE /usuarios/{_id} -> exclusão de usuário

Recurso /produtos

- GET /produtos -> listagem de produtos
- GET /produtos/{_id} -> consulta de produto por ID
- POST /produtos -> cadastro de novo produto (requer usuário administrador autenticado)
- PUT /produtos/{_id} -> edição de produto (requer usuário administrador autenticado)
- DELETE /produtos/{_id} -> exclusão de produto (requer usuário administrador autenticado)

Recurso /carrinhos

- GET /carrinhos -> listagem de carrinhos
- GET /carrinhos/{_id} -> consulta de carrinho por ID
- POST /carrinhos -> criação de carrinho vinculado ao usuário autenticado
- DELETE /carrinhos/concluir-compra -> finalização da compra do carrinho do usuário autenticado
- DELETE /carrinhos/cancelar-compra -> cancelamento da compra do carrinho do usuário autenticado

Regras de negócio e validações

- Unicidade de e-mail no cadastro de usuários
- Controle de permissões via campo "administrador" (true/false)
- Obrigatoriedade e validade do token Bearer nas rotas protegidas
- Restrição de exclusão de usuário ou produto vinculado a um carrinho
- Impedimento de criação de novo carrinho enquanto houver carrinho ativo para o usuário
- Validação de estoque disponível ao adicionar produtos ao carrinho
- Retorno da quantidade de produtos ao estoque ao concluir ou cancelar a compra
- Validação de status codes (200, 201, 400, 401, 404) e mensagens de erro retornadas pela API

2.4. FORA DO ESCOPO

- Testes de interface do front-end (front.serverest.dev) — o foco é a camada de API
- Testes de carga e performance em larga escala
- Testes de segurança avançados (pentest, análise de vulnerabilidades)
- Testes automatizados (o presente plano cobre apenas testes manuais)
- Infraestrutura, deploy e monitoramento da aplicação

3. TIPOS DE TESTES

Tipo de Teste	Descrição	Foco
Funcional	Verificar se os endpoints atendem aos requisitos e contratos esperados	Login, usuários, produtos, carrinhos
Contrato / Schema	Validar a estrutura, tipos de dados e campos obrigatórios das respostas JSON	Respostas de GET, POST, PUT
Negativo / campo inválido	Entradas inválidas, campos obrigatórios ausentes, tipos incorretos, duplicidade	Cadastro de usuário, produto e login
Regras de negócio	Validar restrições específicas da aplicação (estoque, vínculos, permissões)	Carrinho, exclusão de usuário/produto vinculado
Autenticação e autorização	Validar acesso com/sem token, token inválido/expirado e permissões de admin	Rotas protegidas (produtos, carrinhos)
Exploratório	Testes sem roteiro fixo, buscando comportamentos inesperados	Combinações de filtros, paginação, limites de caracteres
Smoke test	Validação rápida das funcionalidades críticas antes de um ciclo completo	Login, cadastro de usuário, listagem de produtos, carrinho

4. ESTRATÉGIA DE TESTES

4.1. ABORDAGEM

Os testes serão realizados de forma manual, com o auxílio de uma ferramenta de requisições HTTP (Postman), baseados em casos de teste derivados da documentação Swagger da própria ServeRest e de cenários de uso real de uma API de e-commerce. A abordagem combina testes baseados em roteiro, para os fluxos críticos (login, cadastro e carrinho), com testes exploratórios para cobrir cenários alternativos e regras de negócio não previstas inicialmente.

4.2. TÉCNICAS DE DESIGN DE CASOS DE TESTE

- Particionamento de equivalência (ex.: e-mails válidos vs. inválidos, tipos de dados corretos vs. incorretos)
- Análise de valor limite (ex.: quantidade de produtos em estoque, tamanho de campos de texto)
- Tabela de decisão para regras de negócio (ex.: usuário administrador vs. não administrador ao cadastrar produto)
- Teste de contrato/schema (validação da estrutura das respostas JSON)
- Casos de uso (fluxo completo: cadastro de usuário → login → cadastro de produto → adicionar ao carrinho → concluir compra)

4.3. PRIORIZAÇÃO

Prioridade	Critério	Exemplos
Alta (Crítica)	Essenciais para o fluxo de compra	Login, cadastro de usuário, criação de carrinho, conclusão de compra
Média	Afetam o uso da API, mas não bloqueiam o fluxo principal	Cadastro/edição de produto, filtros e paginação de listagens
Baixa	Funcionalidades secundárias ou de consulta	Consulta de usuário/produto por ID, mensagens de validação de campos opcionais

4.4. CRITÉRIOS DE ENTRADA

- Ambiente <https://serverest.dev> acessível
- Documentação Swagger da API revisada
- Casos de teste elaborados e revisados
- Ferramentas de gestão e de requisição (Postman) configuradas

4.5. CRITÉRIOS DE SAÍDA

- 100% dos casos de teste de alta prioridade executados
- Mínimo de 90% dos casos de teste totais executados
- Todos os defeitos críticos e de alta prioridade documentados e reportados
- Relatório de execução finalizado

4.6. CRITÉRIO DE SUSPENSÃO E RETOMADA

Os testes serão suspensos caso o ambiente <https://serverest.dev> fique indisponível, a base de dados seja reiniciada durante um ciclo de execução (o ambiente online remove os dados cadastrados diariamente) ou um defeito bloqueante impeça a continuidade das funções principais (login, cadastro ou carrinho). A retomada ocorre assim que o ambiente e essas funções sejam restabelecidos.

5. AMBIENTE DE TESTES E FERRAMENTAS

Categoria	Ferramenta/Recurso
Gestão de casos de teste	Word / Excel
Gestão de defeitos	Excel
Requisições HTTP	Postman (principal) / Insomnia (alternativa)
Documentação da API	Swagger — https://serverest.dev
Evidências	Prints de tela e coleções de request/response (JSON)
Dados de teste	Massa gerada manualmente e https://www.4devs.com.br/

6. CRONOGRAMA

NÃO POSSUI POR SE TRATAR DE UM PORTFÓLIO

7. PAPÉIS E RESPONSABILIDADES

Papel	Responsabilidade
Analista de Teste	Elaborar casos de teste, executar testes, registrar defeitos e reportar resultados

8. ENTREGÁVEIS

- Plano de Testes
- Casos de teste detalhados por endpoint

- Coleção de requisições (Postman)
- Relatório de defeitos